

FISA DE LUCRU. FISIERE TEXT IN PYTHON

Disciplina: Informatica | Clasa: a IX-a | Limbaj utilizat: Python

Continuturi vizate	Fisiere text: caracteristici, mecanisme citire/scriere, sfarsit de fisier. Functii si metode: <code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code> . Clasa <code>TextIOWrapper</code> .
Activitati vizate	Completarea de cod lacunar, generarea de numere aleatoare, tratarea cazurilor limita (fisier lipsa/gol), compararea eficientei de citire.
Obiective	Elevul deschide, citeste, scrie si inchide fisiere text, genereaza date statistice si evalueaza corectitudinea programului pentru cazuri limita.
Durata recomandata	50 minute

1. Retinem ideea principala

Lucrul cu **fisiere text** ne permite sa salvam datele permanent, spre deosebire de variabilele din memorie, care se sterg la inchiderea programului. In Python, lucrul cu fisiere este gestionat implicit prin clasa predefinita `TextIOWrapper`.

Pentru a comunica cu un fisier, parcurgem mereu **3 etape obligatorii**:

1. **Deschiderea fisierului**: Folosim functia `open("nume_fisier.txt", "mod_acces")`.
2. **Transferul de date**: Citim datele din fisier sau scriem date in el.
3. **Inchiderea fisierului**: Folosim metoda `close()` pentru a elibera memoria si a salva fizic modificarile pe disc.

Moduri de acces de baza:

- "r" (read) - Deschide fisierul pentru **citire**. Daca fisierul nu exista, programul va da eroare.
- "w" (write) - Deschide fisierul pentru **scriere**. Creeaza fisierul, iar daca acesta exista deja, **ii sterge continutul anterior**.
- "a" (append) - Deschide fisierul pentru **adaugare**. Scrie noile date la finalul continutului deja existent.

Timpul de executare si "Sfarsitul de fisier" (EOF):

Citirea datelor dintr-un fisier text este extrem de rapida in comparatie cu citirea datelor de la tastatura (`input()`), deoarece se elimina timpul de asteptare al reactiei umane.

Conceptul de **EOF (End of File)** reprezinta momentul in care indicatorul a ajuns la finalul documentului si nu mai exista caractere de citit. In Python, cand citim cu `read()`, ajungerea la EOF returneaza un sir de caractere gol `""`.

2. Exemple rezolvate

A. Generarea si scrierea de numere aleatoare intr-un fisier

Pentru aplicatiile practice, de multe ori trebuie sa generam noi datele de intrare.

```
import random
```

Etapa 1: Deschidem fisierul pentru scriere

```
f_out = open("numere_generate.txt", "w")
```

Etapa 2: Transferul de date

for i in range(5):

numar = random.randint(10, 99)

Atentie! Metoda write() accepta DOAR date de tip string (str)

\n adauga o trecere la rand nou

f_out.write(str(numar) + "\n")

Etapa 3: Inchidem fisierul

f_out.close()

B. Tratarea cazurilor limita (Fisier inexistent sau gol)

Pentru a evalua corectitudinea unui program, trebuie sa testam ce se intampla cand lucrurile merg prost (ex: fisierul lipseste).

try:

f_in = open("fisier_test.txt", "r")

continut = f_in.read()

Testam cazul unui fisier complet gol

if continut == "":

print("Fisierul a fost gasit, dar este complet gol!")

else:

print("Datele sunt:", continut)

f_in.close()

except FileNotFoundError:

Testam cazul in care fisierul nu exista

print("Eroare severa: Fisierul cerut nu a fost gasit pe disc!")

3. Exercitii diversificate

A. Completeaza spatiile libere

1. Functia folosita in Python pentru a asocia un fisier text unei variabile din program este _____.
2. Pentru a citi intregul continut al unui fisier si a-l stoca intr-un singur sir de caractere, folosim metoda _____.
3. Apelarea metodei _____ este obligatorie la finalul lucrului pentru a garanta ca modificarile facute prin scriere nu se pierd.
4. Daca dorim sa adaugam rezultatele de astazi la finalul fisierului creat ieri, vom deschide fisierul cu modul de acces "_____".

B. Adevarat sau fals

1. Metoda write() permite transmiterea directa a unui numar intreg ca argument, exact la fel ca functia print() (de exemplu: f.write(100)). (A / F)

2. Executia unui program care prelucreaza 100.000 de numere citite dintr-un fisier text va fi mult mai rapida decat daca un utilizator ar tasta manual acele numere in consola. (A / F)
3. Modul de acces "w" este periculos daca este folosit necorespunzator, deoarece poate suprascrie date importante existente in fisier inainte de deschidere. (A / F)
4. La citire, ajungerea la "sfarsit de fisier" (EOF) opreste brusc si forteaza inchiderea programului Python printr-o eroare de sistem. (A / F)

C. Exerciitiu de lucru

Cod lacunar: Prelucrarea unui numar

Presupunem ca fisierul intrare.txt contine un singur numar intreg pozitiv. Completeaza codul pentru a citi acest numar, a-i calcula patrutul si radacina patrata, si a scrie rezultatele intr-un alt fisier, numit iesire.txt.

```
import math
```

```
# --- CITIREA ---
```

```
f_in = ____("intrare.txt", " ____ ")
```

```
valoare_sir = f_in.read()
```

```
n = int(valoare_sir) # conversie la intreg
```

```
f_in.____()
```

```
# --- PRELUCRAREA ---
```

```
patrat = n ** 2
```

```
radacina = math.sqrt(n)
```

```
# --- SCRIEREA ---
```

```
f_out = ____("iesire.txt", " ____ ")
```

```
f_out.write("Patratul este: " + str(____) + "\n")
```

```
f_out.write("Radacina este: " + str(____) + "\n")
```

```
f_out.____()
```

Barem orientativ / indicatii

- **A:** 1. open(); 2. read(); 3. close(); 4. a (append).
- **B:** 1-F (metoda write() primeste exclusiv siruri de caractere / stringuri; numerele trebuie convertite cu str()); 2-A (este demonstratia diferentei de performanta I/O); 3-A; 4-F (EOF in Python la apelarea read() nu genereaza o eroare, ci pur si simplu returneaza un string gol).
- **C. Cod lacunar:** open, "r", close(), open, "w", patrat, radacina, close().